



Après le vibe coding :
garder le volant pour une
réaliser une Power Apps
(avec Code Apps)



Julien CHABLE



aMP Montpellier
24/09/2026





L'agent écrit le code. Qui contrôle ?

- Un agent réussit ~6 tâches sur 10 au 1er essai (pass@1 $\approx$ 60 %)
- ... mais n'est reproductible que dans ~1 cas sur 4 (~25 %)
- La vitesse n'est pas la fiabilité
- L'enjeu : garder le contrôle quand l'agent génère votre PowerApp/application

MERCI à nos SPONSORS !

PowerSyncPro
Migration Agent

Sponsor Gold


Modernize

Sponsor Gold

sopra  steria

Partenaire Organisateur

Prochains événements aMP – 2nd semestre 2026





Julien CHABLE

Consultant & Développeur

- IA appliquée au développement logiciel
- Consultant/CEO

Coderise : www.coderise.fr

Agenda

“Après le vibe coding : garder le volant pour une réaliser une Power Apps (avec Code Apps) »

1. Du vibe coding à l'agentic engineering
2. Les trois garde-fous : spec-driven, TDD & evals
3. Code Apps : générer votre Power App (démon)
4. Recul critique & à emporter

Du vibe coding à l'agentic engineering



2025 → 2026 : un virage en un an

- Fév. 2025 ➔ « vibe coding » (Karpathy) : parler au LLM, oublier le code
 - Élu mot de l'année 2025 (Collins)
- 2026 ➔ « agentic engineering » : orchestrer des agents, sous contrôle

Software 1.0

Du code écrit à la main

Le développeur écrit
chaque ligne



Software 2.0

Des modèles entraînés

On donne des exemples,
le réseau apprend les
règles



Software 3.0

Du langage naturel

2025 : vibe coding
2026 : agentic engineering

Le paysage des méthodes « -Driven »

- **SDD** (*Spec-Driven Development*) — la spec = source de vérité (niveau feature / système)
- **TDD** (*Test-Driven Development*) — un test d'abord (niveau unité)
- **EDD** (*Eval-Driven Development*) — des evals continues (fiabilité en production)
- Et aussi : *BDD / ATDD (Given/When/Then)*, *IDD / Prompt-Driven*, ...

Elles ne s'opposent pas, elles s'empilent

Intent / spec · en amont



Tests · au milieu

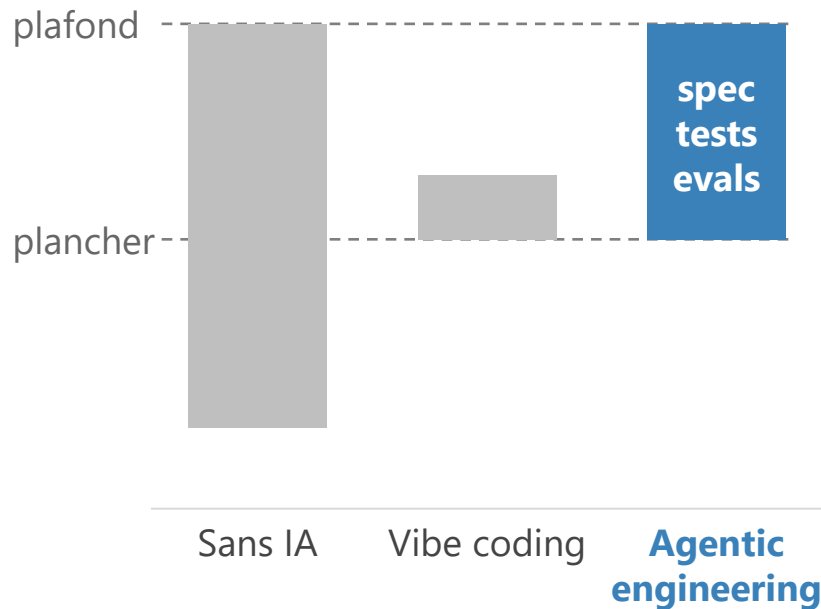


Evals · en aval, en prod

La thèse : trois garde-fous

- Le vibe coding relève le plancher...
- ... l'agentic engineering préserve le plafond
- Mode d'échec à l'échelle : dette, failles, zéro supervision
- La réponse : la spec en amont + les tests au milieu + les evals en prod

Qualité du code livré

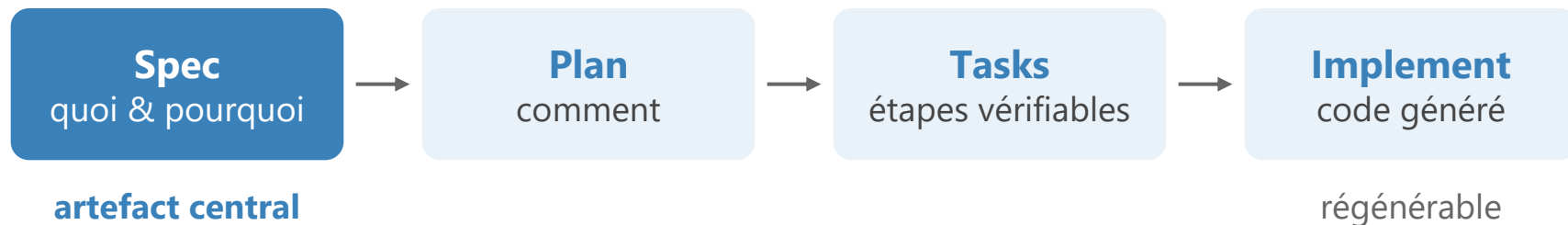


Les trois garde-fous



Garde-fou #1 — la spécification (SDD)

- La spec devient l'artefact central ; le code, un output généré
- Autre outils « Spec driven » :
 - GitHub Spec Kit : Spec → Plan → Tasks → Implement
 - Amazon Kiro (IDE agentic)
 - BMAD (agents en rôles)

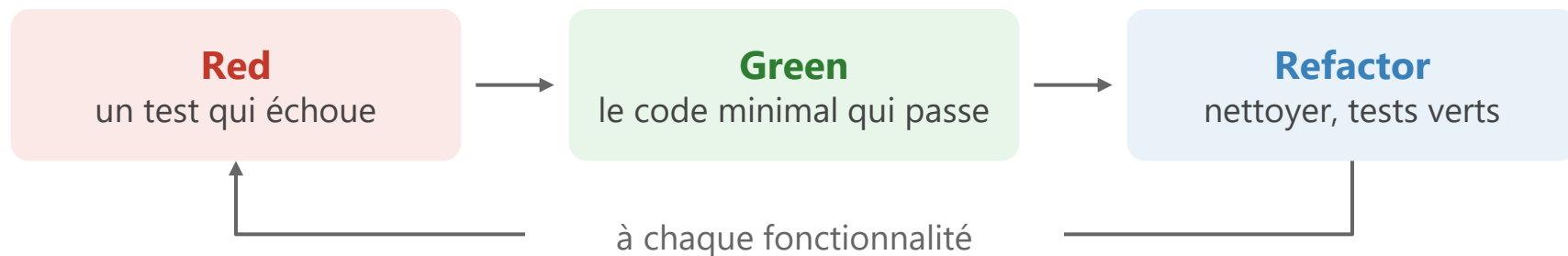


La constitution du projet

- CLAUDE.md / AGENTS.md à la racine, lu avant chaque prompt
- Nos règles Code Apps : React + TypeScript, SDK @microsoft/power-apps
- Tests d'abord (Vitest), connecteurs typés, pas de secret en dur
- Un fichier d'instructions Code Apps existe déjà (awesome-copilot)
- La brique de « context engineering »

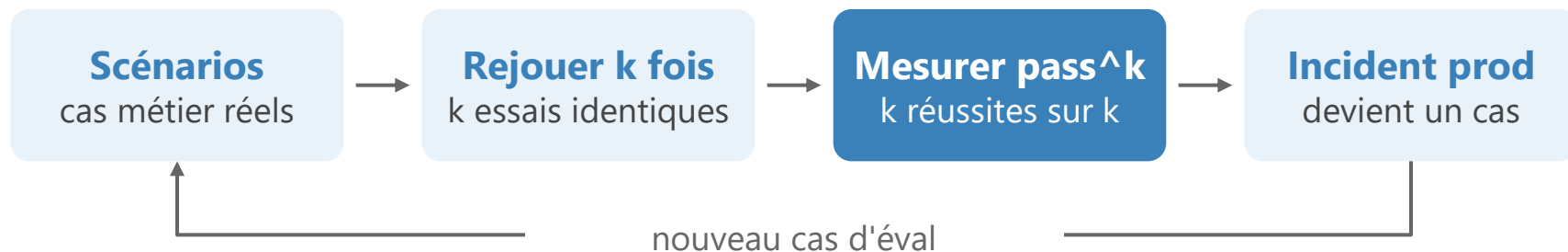
Garde-fou #2 — les tests (TDD)

- La définition du « fini » / « done » devient exécutable
- Donc non hallucinable : l'agent ne valide pas ses propres bugs
- Cycle Red → Green → Refactor
- Anti-cheat : exiger la sortie brute des tests avant le « vert »



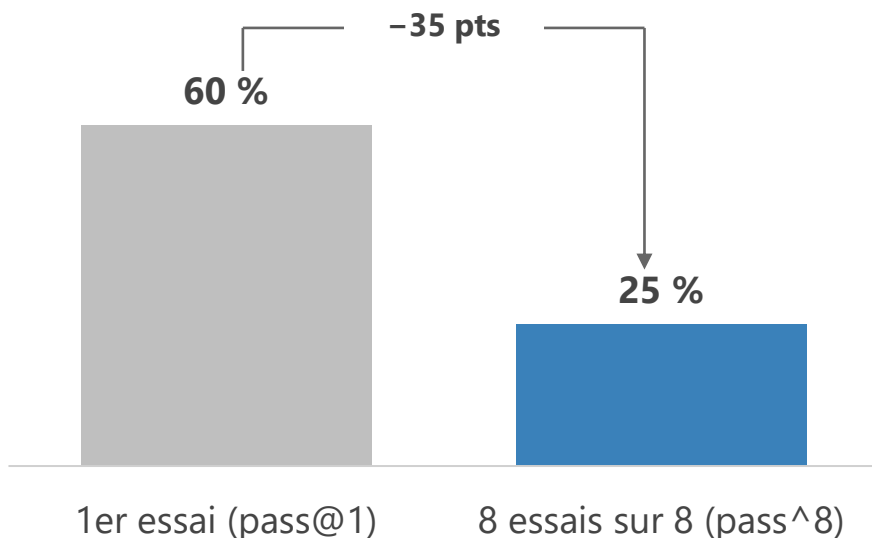
Garde-fou #3 — les evals (EDD)

- La fiabilité se mesure sur des essais répétés, pas sur une démo réussie
- Un jeu d'évals métier, rejoué à chaque changement de modèle, de prompt ou de spec
- En prod : chaque incident devient un nouveau cas d'éval



60 % au premier essai, 25 % sur 8 essais sur 8

Taux de réussite d'un agent sur la même tâche



- **Une démo réussie** ne dit rien de la fiabilité
- Rejouer chaque scénario **plusieurs fois** avant de livrer
- Suivre **pass^k**, pas pass@1 : l'utilisateur ne relance pas

Code Apps : générer votre Power App



Code Apps : coder votre Power App

- Une SPA React / TypeScript hébergée dans Power Platform
- Accès typé à 1500+ connecteurs, Dataverse, zéro code d'auth
- Auth, hébergement et sécurité gérés par la plateforme
- Outillé en CLI : `pac code / pa` (init → run → push)
- GA février 2026 → le pro-code entre dans Power Apps

Applications de code Power Apps

Autorisez les utilisateurs à importer les applications qu'ils ont créées en dehors de Power Apps et à les exécuter dans Power Apps. Remarque : si ce paramètre est désactivé, ces applications ne fonctionneront plus. Vérifiez que l'application de code provient d'une source fiable. [En savoir plus](#) 

Activez les applications de code



Activé

L'outil : Superpowers

- Méthodologie complète en skills auto-déclenchés (Obra / J. Vincent)
- Le loop : brainstorm (spec) → plan → sous-agents → TDD → revue → merge
- Couvre SDD (design doc), TDD strict (RED-GREEN-REFACTOR) et la revue
- Mode SDD : contrôleur / implémenteur / reviewer séparés
- Cross-harness (Claude Code, Codex, Cursor...) ➔ ici, en Claude Code



Démo — la feuille de route

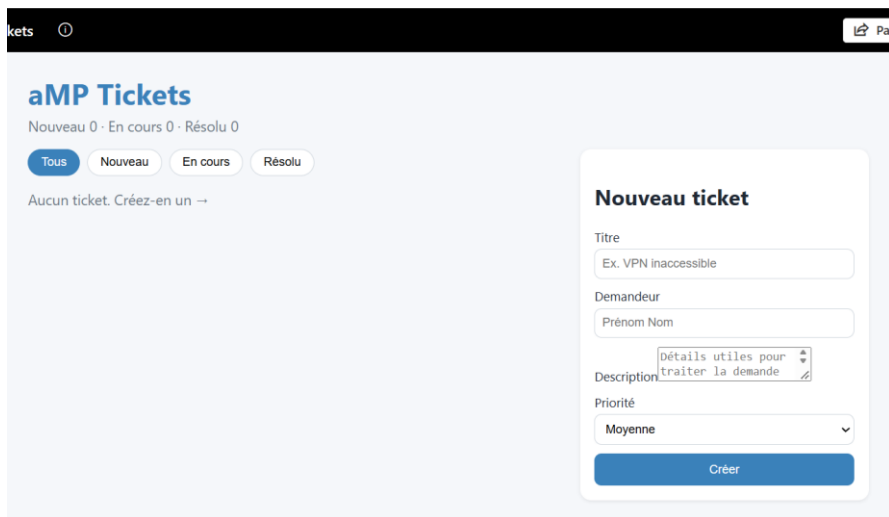
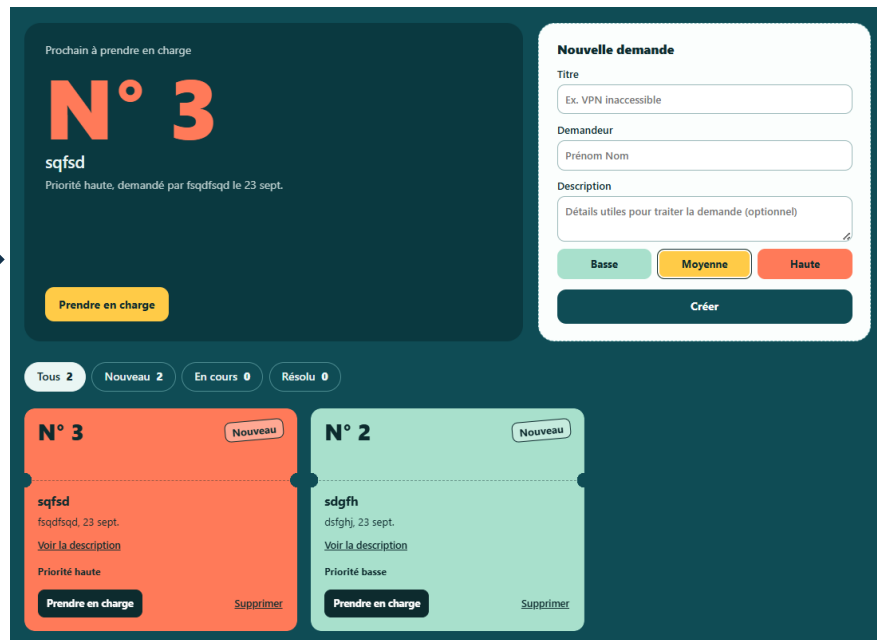
- 0 · Constitution : Claude.md/AGENTS.md/ ... → règles Code Apps (React/TS + SDK)
- 1 · brainstorming → design doc (la spec de la Power App)
- 2 · writing-plans → implementation doc (STD avancé)
- 3 · subagent-driven-development + TDD (Vitest, RED→GREEN)
- 4 · requesting-code-review → pac code push (déploiement)
- Le cas : une PowerApp CRUD (gestion de tickets) sur SharePoint

Ce qu'il faut regarder

- Le test ROUGE écrit avant le code (Vitest)
- La sortie brute des tests (pas de triche)
- Le sous-agent qui relit contre le plan
- La Power App générée : belle, connectée, déployée (pac code push)

Une passe UI quand même ... ça ne fait pas de mal !

Utiliser le plugin *frontend-design* ou Claude Design (n'essayez même pas avec Copilot 🤖)

The image shows a web interface for 'aMP Tickets'. At the top, there's a header with 'tickets' and a user profile icon. Below the header, the title 'aMP Tickets' is followed by a status bar: 'Nouveau 0 · En cours 0 · Résolu 0'. There are four filter buttons: 'Tous' (selected), 'Nouveau', 'En cours', and 'Résolu'. Below the filters, it says 'Aucun ticket. Créez-en un →'. The main content is a 'Nouveau ticket' form. It has fields for 'Titre' (with the example 'Ex. VPN inaccessible'), 'Demandeur' (with 'Prénom Nom'), 'Description' (with a placeholder 'Détails utiles pour traiter la demande'), and 'Priorité' (set to 'Moyenne'). A 'Créer' button is at the bottom of the form.The image shows a web interface for 'aMP Tickets' displaying a list of tickets. The header is dark blue with the text 'Prochain à prendre en charge'. Below the header, there's a large orange card for 'N° 3' with the title 'sqfsd', priority 'haute', and a 'Prendre en charge' button. To the right, there's a 'Nouvelle demande' form with fields for 'Titre', 'Demandeur', 'Description', and 'Priorité'. Below the main card, there's a list of tickets. The first card is orange for 'N° 3' with title 'sqfsd', priority 'haute', and a 'Prendre en charge' button. The second card is green for 'N° 2' with title 'sdgfh', priority 'basse', and a 'Prendre en charge' button. At the bottom, there's a status bar: 'Tous 2 · Nouveau 2 · En cours 0 · Résolu 0'.

CONCLUSION

Ce qu'on garde
Ce qu'on jette
Ce qu'on emporte



Ce que les garde-fous ont évité

La dérive de l'agent

Évité par
la spec (#1)

Le sur-code / le code inventé

Évité par
la spec et les tests (#1, #2)

Le bug auto-validé et les code smells

Évité par
les tests et les evals (#2, #3)

Résultat : une Power App testée, prête pour la prod ! (ou presque)

Quand s'en passer

Il vaut mieux s'en passer

- Prototypes jetables, exploration
- Besoin simple : une Canvas app suffit souvent
- Coût amont + « semantic gap »

L'adopter

Brille sur les Power Apps complexes, durables, en prod

À emporter



- Le pipeline : constitution → spec → plan → TDD → revue → déploiement
- Le « DONE » : une Power App fonctionnelle, belle, conforme
- Adopter le spec-driven... ou savoir s'en passer quand il faut

Le futur : low-code et Code Apps convergent

Low-code

Canvas, model-driven

- Reste le format **le plus utilisé**
- **Co-création avec des agents** en langage naturel (GA 2026)
- Pour : besoin simple, maker métier

Code Apps

React / TypeScript

- GA depuis le **5 février 2026**
- UI libre, Git, **tests et CI/CD**
- Pour : app complexe, durable, équipe dev + agents

Prototyper en vibe coding, puis industrialiser en Canvas ou en Code Apps.

**MERCI !
THANK YOU!**

**pour votre participation
et vos retours**

Scan me →



Après le vibe coding : garder le volant pour une réaliser une
Power Apps (avec Code Apps)